

## 2-Week Sprint Battle Plan

**Context for Claude Code:** This is a robotaxi HMI prototype with three surfaces communicating over WebSocket through a Node relay server: `orchestrator.html` (vehicle-state simulator on a phone), `trackpad.html` (iPad armrest input), `display.html` (in-vehicle screen). After a successful CEO demo ride, this sprint addresses bugs observed during the ride, CEO requests, and architectural cleanup. The codebase has evolved since this plan was written — real Mapbox + USB GPS replaced earlier mock navigation. **Verify the current state of any file before changing it.** If a task's premise no longer matches the code, stop and report back rather than proceeding on stale assumptions.

**Working principle:** Small, surgical changes. No rewrites. Behavior-preserving refactors only. Each task below is a discrete session — finish one, verify it works, commit, then start the next.

---

### Task 1 — Fix precondition state machine race

**File:** `cursor-trackpad/public/js/trackpad.js` **Estimated effort:** ~1 hour **Priority:** Highest. This is the bug the CEO may have noticed.

#### Problem

`checkPreconditions()` fires `startTransition()` after a 1-second delay whenever both door and seatbelt are checked. There are two race conditions:

1. **No idempotency guard.** If both preconditions go true, then one is toggled off and back on, `startTransition()` fires a second time, replaying the entire intro animation on top of itself.
2. **The 1s timer can't be cancelled.** If both preconditions are checked (timer scheduled), then one is unchecked during the delay window, the timer still fires.

#### What to do

1. Read the current `checkPreconditions()` and surrounding state. Confirm the bug shape still matches the description above. **If preconditions are now received as a state snapshot rather than per-action toggles, stop and report the new shape — the fix below assumes per-action messages.**
2. Add two instance fields in the constructor: `this.transitionStarted = false` and `this.preconditionTimer = null`.
3. Rewrite `checkPreconditions()` so it:
  - Returns immediately if `this.transitionStarted` is true.
  - Cancels any existing `this.preconditionTimer` before checking.

- Only schedules a new timer if both preconditions are currently true.
  - Inside the timer callback: re-checks both preconditions are still true (they may have flipped during the 1s delay), sets `transitionStarted = true`, then calls `startTransition()`.
4. In the WebSocket `ride-end` handler (or wherever ride teardown lives), reset `this.transitionStarted = false` and clear the timer when the ride ends/resumes, so a subsequent ride can transition again.

### Acceptance criteria

- Toggling door on, seatbelt on, then door off within 1s: no transition fires.
- Toggling door on, seatbelt on (transition fires), door off, door on again: no second transition.
- After ride-end → ride-resume, the precondition flow can fire again normally.
- No console errors. No new timers leaked.

### Out of scope

Do not refactor the broader event-vs-state model in this task. That's intentionally deferred to a future sprint. Keep the change local to `checkPreconditions()` and the constructor.

---

## Task 2 — WebSocket heartbeat + reconnect-on-stale

### Files:

- `cursor-trackpad/server/server.js`
- `cursor-trackpad/public/js/trackpad.js`
- `cursor-trackpad/public/js/display.js`
- `cursor-trackpad/public/orchestrator.html` (inline script)

**Estimated effort:** ~3 hours **Priority:** High. Suspected cause of the iPad PWA cursor-loss-after-45s issue and a class of silent-failure bugs.

### Problem

Sockets can hang at the TCP layer without `onclose` ever firing. When this happens, `send()` silently no-ops because `readyState !== OPEN`, and `attemptReconnect()` never runs because no close event fires. The UI keeps working locally but messages stop being delivered.

The 45-second iPad PWA issue is consistent with iOS Safari closing idle WebSocket connections in PWA standalone mode without the JS layer being reliably notified.

### What to do

## Server side (`server.js`):

1. After `wss.on('connection', ...)`, set `ws.isAlive = true` on the new socket.
2. Register a `pong` handler on the socket: `ws.on('pong', () => { ws.isAlive = true })`.
3. Add a single server-wide interval (e.g. 15 seconds) that iterates `wss.clients`: for each client, if `ws.isAlive === false` call `ws.terminate()`, otherwise set `ws.isAlive = false` and call `ws.ping()`. Clean up the interval on `wss.on('close', ...)`.

## Client side (each of `trackpad.js`, `display.js`, `orchestrator inline script`):

4. The browser `WebSocket` API auto-responds to ping frames with pong — no JS needed for that. But the browser doesn't expose a way to detect a hung connection. So implement an **application-level heartbeat** as well:
  - On `onopen`, start a `setInterval` (e.g. every 10 seconds) that sends `{type: 'heartbeat'}`.
  - On every received message, update a `this.lastMessageAt = Date.now()` field. (Server's pings happen below the JS layer but app-level traffic counts as activity.)
  - Run a watchdog `setInterval` (every 5 seconds) that checks if `Date.now() - this.lastMessageAt > 30000`. If so, treat the connection as stale: call `ws.close()` (which triggers `onclose` → `attemptReconnect`).
  - Clean up both intervals on `onclose` so they don't accumulate across reconnects.

## Server side, message handler:

5. In the message handler, ignore `{type: 'heartbeat'}` messages at the top — don't relay them. Optionally echo back `{type: 'heartbeat-ack'}` so clients update `lastMessageAt` on the round-trip even if no other surface is sending traffic. This matters because if e.g. only the orchestrator is connected and idle, the trackpad might otherwise watchdog itself.

## Acceptance criteria

- Disconnect the iPad's network for 60 seconds, reconnect: trackpad detects the stale connection, reconnects, and resumes sending input within ~30s.
- With all three surfaces connected and idle for 5+ minutes, no surface watchdogs itself off (heartbeat-ack keeps `lastMessageAt` fresh).
- Server logs show stale clients being terminated rather than accumulating in `wss.clients`.
- The 45-second iPad PWA issue should be either fixed or unchanged. **If unchanged, report that — it points to a deeper iOS PWA issue (pointer arbitration, not WebSocket) and needs a different fix path.**

## Out of scope

- Don't add reconnection backoff tuning beyond what `attemptReconnect()` already does.
  - Don't add server-side per-client logging unless debugging requires it.
- 

## Task 3 — Diagnose Spotify LAN failure

**Files:** Read-only investigation across `server.js`, `display.js`, `spotify-link.html`, `spotify-auth.html`, `spotify-mock.html`, `spotify-player.js`. **Estimated effort:** ~2-4 hours **Priority:** High but unblocked — this is a diagnosis task, no code changes yet.

### Problem

During the demo, Spotify didn't work in the LAN environment. The reported reason was "couldn't get the auth key due to unsafe HTTP." But the server has a working PKCE-based session-handoff flow (`POST /spotify/session`, QR → phone → token relay), which means the architecture *should* tolerate the display being on HTTP as long as the phone reaches HTTPS for the actual OAuth.

Before assuming infra has to fix HTTPS, find out exactly what broke.

### What to do

#### 1. Trace the full Spotify auth flow end-to-end by reading the code:

- Where is the QR code generated and what URL does it encode?
- Where does the phone go after scanning? Is it `https://accounts.spotify.com/authorize` directly, or via an intermediate page on the LAN server?
- If intermediate: what URL is the phone hitting? Does Spotify's OAuth redirect URI resolve in the LAN environment?
- At what step does the failure occur — QR scan, phone reaching the auth page, OAuth redirect back, token POST to server, or token relay to display?

#### 2. Identify the actual failure point. Common candidates:

- QR encodes `localhost` or `127.0.0.1`, which the phone can't reach.
- QR encodes the LAN IP (e.g. `192.168.x.x`) but Spotify's registered redirect URI doesn't match it.
- Phone hits the LAN server over HTTP, browser blocks the OAuth flow because Spotify requires HTTPS for redirect URIs.
- Token relay works but client-side Spotify Web Playback SDK refuses to load over HTTP.

3. **Write a one-page diagnosis** with: which step fails, why, and three options ranked by effort:

- **Cheap fix** if any (e.g. encoding the LAN IP correctly, configuring redirect URI).
- **Medium fix** (e.g. running the LAN server behind a self-signed HTTPS cert, or a reverse-proxy with Let's Encrypt for a `*.local` hostname).
- **Expensive fix** (full HTTPS infra for the LAN environment).

### Acceptance criteria

- Diagnosis report identifies the precise failure step.
- At least one of the three options is genuinely actionable inside the 2-week sprint without depending on another team.

### Out of scope

- Don't implement the fix in this task. The fix lands in a follow-up task once the diagnosis is in.

---

## Task 4 — Extract cursor system into `cursor.js`

### Files:

- New: `cursor-trackpad/public/js/cursor.js`
- Modified: `cursor-trackpad/public/js/display.js`
- Modified: `cursor-trackpad/public/display.html` (script tag added)

**Estimated effort:** ~1.5 days **Priority:** Medium-high. Unblocks all future cursor work without touching `Display`.

### Problem

The cursor system — custom DOM, render loop with velocity-based skewing, smooth scroll lerp, hover detection across `data-cursor-text` / `data-cursor-img` / `data-cursor-video` / `data-cursor-stick` / `data-magnetic-btn` / `data-cursor`, the magnetic button physics, custom `mouseenter` / `mouseleave` dispatch, the `.is-hovered` class management — is currently ~400 lines inside the `Display` class. It's not a missing system; it's a buried one.

Extracting it makes future cursor work (CEO request: more cursor states, scroll indicator, tooltip-style hints) low-risk and isolated.

### What to do

1. **Read the current `Display` class end-to-end** before touching anything. Identify which

fields, methods, and DOM nodes belong to the cursor and which belong to display logic. Some are entangled — `enterCursorMode()`, `activateCursor()`, the cursor-mode timing, the `cursor-ready` event — note these as boundary cases.

2. **Define the `Cursor` module's public API.** Suggested shape:

```
Cursor.init(options)           // creates DOM + styles, starts render loop
Cursor.show() / Cursor.hide()
Cursor.setEnabled(bool)       // disables hover/click handling without
hiding
Cursor.setMode('arrow' | 'round')
Cursor.moveBy(dx, dy)         // called by display's WebSocket move handler
Cursor.click(button)          // called by display's WebSocket click
handler
Cursor.scroll(dx, dy)         // called by display's WebSocket scroll
handler
Cursor.doubleClick()
Cursor.setHoverableSelector(str) // override default selector
Cursor.on(event, handler)      // 'activated', 'hover', 'click' etc.
```

Confirm this shape with the human before implementing — there may be constraints from the trackpad side that affect the API.

3. **Move the code.** Behavior must be identical. No optimizations, no new features, no fixes (even small ones — log them as TODOs and address separately). The cursor module owns: DOM creation, the inline `<style>` injection, render loop, `checkHoverState`, `setCursorImage`, `setCursorVideo`, `clearCursorMedia`, `updateMagneticButton`, `simulateHover`, `findScrollableParent`, all the `data-cursor-*` handling.
4. **Keep `Display` responsible for:** WebSocket lifecycle, message routing, phase choreography, the GSAP timelines (`playDarkStateTimeline`, `playDrivingViewTimeline`, `playRideEndTimeline`, `playRideResumeTimeline`), `enterCursorMode`/`activateCursor` (which now call into `Cursor`), `handleDebug`. The cursor instance lives as `this.cursor = new Cursor(...)` on the Display.
5. **Write a short reference doc** (`cursor-trackpad/public/js/cursor.md`) or a comment header in `cursor.js`) listing every supported `data-cursor-*` attribute and what it does. The CEO ask for "more cursor states" implies more attributes will be added; a central reference prevents drift.

## Acceptance criteria

- All current cursor behaviors work identically: velocity skew, smooth scroll, magnetic, image preview, video preview, text labels, stick, magnetic button physics, custom hover events.

- `display.js` no longer contains cursor DOM creation, the cursor render loop, or `checkHoverState`.
- The cursor module loads cleanly on `display.html` and could in principle be loaded on other surfaces (don't actually load it elsewhere yet).
- A diff-friendly commit history: one commit creates `cursor.js` with the moved code, a second updates `display.js` and `display.html` to use it.

## Out of scope

- No new cursor states or features. That's Task 6.
  - Don't extract anything else from `Display` (the timelines, etc.) even if tempted.
  - Don't load `cursor.js` on the trackpad or orchestrator pages.
- 

## Task 5 — Adopt GSAP Flip for shared-element transitions

### Files:

- `cursor-trackpad/public/display.html` (add Flip plugin script)
- `cursor-trackpad/public/js/display.js` (refactor `playRideEndTimeline`, possibly `playRideResumeTimeline`)

**Estimated effort:** ~1.5 days **Priority:** Medium-high. Architectural fix for "components disappear and reappear" instead of flowing.

### Problem

`playRideEndTimeline` currently implements a manual shared-element transition for the user avatar moving from the user-card position to the inline ride-end position: it measures both rects, builds a temporary `floating-avatar` div, animates it, then removes it and reveals the inline avatar. This works but every continuity transition has to be hand-built this way. GSAP Flip is the standard primitive for exactly this problem.

### What to do

1. **Verify GSAP Flip is available** in the version of GSAP currently loaded. Flip is part of GSAP's bonus plugins and may need to be added to the script tag in `display.html`. If the version doesn't support Flip, stop and report — the human can decide whether to upgrade.
2. **Refactor the avatar handoff** in `playRideEndTimeline`:
  - Before changing visibility: capture the layout state of the source element (`#user-card` avatar region) with `Flip.getState()`.
  - Hide the source, reveal the destination (`#ride-end-avatar`), in the same frame.

- Call `Flip.from(state, { duration, ease, ... })` to animate the destination from where the source was to where it ends up naturally.
  - Remove the manual `floating-avatar` div, the `getBoundingClientRect()` measurements, the `Object.assign(floating.style, ...)` setup, and the `floating.remove()` callback. The Flip plugin handles the FLIP animation (First-Last-Invert-Play) for you.
3. **Spot-check the timeline interleaving.** `playRideEndTimeline` orchestrates many tweens at specific time offsets. The Flip animation needs to slot in at the same point and have a similar duration so the rest of the timeline doesn't desync. Use `tl.add(Flip.from(...), offset)` to nest it.
  4. **Identify one more transition that would benefit.** A good candidate is anywhere a component is currently destroyed and recreated across phases. Examples to check: the comm-island → arrived-pill morph (currently done with width animation + opacity crossfade), the home → ride-start chrome handoff. **Pick at most one** for this task. If neither is clean enough, stop at the avatar refactor.
  5. **Note: the `playRideResumeTimeline` currently uses `clearProps` to reset things back.** With Flip, the reverse transition might be expressible as another `Flip.from` call with the source/destination swapped. Worth attempting, but if it's messy, leave the resume timeline as-is for now.

### Acceptance criteria

- The avatar handoff at ride-end animates as smoothly as before (or smoother).
- No `floating-avatar` div in the DOM at any point.
- The ride-end timeline still completes in roughly the same total duration (~2s).
- `playRideResumeTimeline` still correctly resets the state for a subsequent ride-end.
- No GSAP warnings in the console.

### Out of scope

- Don't refactor any timeline that doesn't have a shared-element problem.
- Don't change the visual design — same end states, same easing feel.

## Task 6 — Extend cursor states (after Task 4 is merged)

**File:** `cursor-trackpad/public/js/cursor.js` **Estimated effort:** ~1 day **Priority:** Medium. CEO-facing improvement.



Task 4 must be complete. Do not start this task before the cursor extraction is merged and verified.

## Problem

CEO wants the cursor to be more communicative: contextual labels (already supported via `data-cursor-text` — extend), scroll indicators when scrollable content is hovered, tooltip-style hints that periodically introduce gestures or complex interactions to the user.

## What to do

Confirm scope with the human before implementing. Suggested initial states:

1. **Scroll indicator state.** When the cursor is over a scrollable element (use the existing `findScrollableParent` logic), show a vertical-arrows icon inside the cursor instead of the dot. Class: `.has-scroll`. New CSS, no new JS logic beyond detection.
2. **Hint mode.** A new public API: `Cursor.showHint(text, durationMs)` that briefly displays a tooltip-style label near the cursor with text like "Two-finger scroll to see more" or "Tap and hold to drag." Auto-dismisses after `durationMs`. Used by display logic to teach gestures contextually — e.g. first time the user enters a scrollable area in a session, show "Two-finger scroll" once.
3. **State flag for active interaction.** When `Cursor.click()` is called, briefly add a `.acquiring` class for visual feedback distinct from the existing `.clicking` (which is the press-down feedback). This is a small detail that improves the feel of magnetic acquisition, especially in a moving vehicle.

Do not add more than these three in this task. Each new state requires CSS work and edge-case handling; landing three solid states is more valuable than scaffolding ten.

## Acceptance criteria

- Each new state is documented in the cursor reference (the doc/header from Task 4).
- States compose correctly (e.g. `.has-scroll` + `.magnetic` doesn't break either).
- No regression in existing states.

## Out of scope

- Don't redesign the existing cursor visuals.
  - Don't touch the trackpad side.
-

**Files:** New `cursor-trackpad/public/games-mock.html`, possibly small additions to `display.js`. **Estimated effort:** ~1 day **Priority:** Low. CEO ask but not load-bearing for the next demo. Drop first if the sprint runs over.

## What to do

1. Mirror the structure of `youtube-mock.html` for a `games-mock.html`. It's a placeholder app that lives in the entertainment space — not a real game.
2. Hook it into the app launcher / media panel so it's accessible from the display, with cursor support.
3. Use a single static screen (e.g. a card grid of "coming soon" game tiles) — enough to show the CEO that the slot exists and is reachable. No interactivity beyond cursor hover.

## Acceptance criteria

- Reachable from the display UI via cursor.
- Visually consistent with existing mock apps.
- No actual game logic.

## Out of scope

- Don't build a real game.
  - Don't add a games framework or library.
- 

## Items explicitly deferred to a future sprint

These are flagged so they don't get smuggled into the 2-week scope:

- **Server-as-canonical-state-holder** with state snapshots replacing per-action events. This is the deeper fix for the precondition bug class. Estimated 2 days. Worth doing before adding more surfaces or scope.
- **Analytics / telemetry layer** for the UX research team. Estimated 1-2 days. Easier *after* the state model refactor above (instrument transitions, not events).
- **Decomposition of the rest of `Display`** — the GSAP timelines, the WebSocket layer, the phase choreographer should each be their own modules eventually. Do this gradually as parts are touched.
- **Mapbox token to env config.** The current navigation-map.js (or its successor with real GPS) hardcodes a token. Move to a runtime config endpoint similar to how Spotify Client ID is exposed via `/config`.

---

## How to use this document with Claude Code

- Run **one task per session**. Don't paste the whole document and ask Claude Code to "do everything." Pick the task, paste that section plus this header, and let Claude Code work within scope.
- Each task starts with a **read/verify step** because the codebase has evolved (Mapbox + GPS). If the premise no longer matches, Claude Code should report back rather than proceed.
- After each task, **commit before starting the next**. Easier to revert if something goes sideways.
- For Task 4 in particular (cursor extraction), expect to iterate. The first cut probably won't be clean — review it, request adjustments, then merge.